

CS246 Spring 2024 Design Document - CC3K

Overview

In software development, achieving high cohesion and low coupling is essential for creating a well-organized, maintainable, and scalable system. For my CC3K game project, I accomplished this by effectively utilizing polymorphism and inheritance, and strategically applying design patterns. This strategy ensured that each component remained focused on its core responsibilities, while enhancing overall flexibility and minimizing dependencies throughout the system. The system is mainly composed of a game engine represented by the `Game` class, two three-level inheritance hierarchy (one for characters and the other one for items) and a decorator pattern to dynamically modify player attributes without adding level specific attributes in player's class.

Main Game Components

Game:

The core class managing the overall game state. It controls the game flow, handles user inputs, updates the game board, and manages interactions between characters and items. Furthermore, it is responsible for initializing the game environment, spawning characters and items, and advancing the game through turns or levels.

Three-level inheritance hierarchy for characters:

`Character`: An abstract base class representing player characters and enemies in the game, providing common attributes and methods.

`Player` and `Enemy`: Derived classes include specific types of characters such as players and enemies, each with unique behaviors and attributes. These classes inherit from the `Character` class.

`Player`: Represents the player-controlled character, with subclasses for different races and abilities.

`Enemy`: Represents various hostile characters in the game, with subclasses for different enemy types and behaviors.

`Shade`, `Goblin`, `Troll`, `Drow`, `Vampire`: Concrete subclasses of the `Player` class representing player types with distinct stats and abilities.

`Human`, `Dwarf`, `Elf`, `Orc`, `Merchant`, `Halfling`: Concrete classes subclasses of the `Enemy` class representing enemy types with distinct behaviors and abilities.

Three-level inheritance hierarchy for items:

`Item`: An abstract base class representing the items in the game. Items include gold and potions, each with specific properties and effects. The base class provides common functionality for items, such as their position on the board and interactions with characters.

`Potion` and `Gold`: Derived classes that inherit from the `Item` class.

`Potion`: Represents consumable items that temporarily alter the player's attributes.

`Gold`: Represents various types of treasure that can be collected.

`RH`, `BA`, `BD`, `PH`, `WA`, `WD`: Concrete subclasses of `Potion` class representing different potions available in the game. Each with specific effects on the player.

`DragonHoard`, `MerchantHoard`, `HumanPile`, `NormalPile`, `SmallPile`: Concrete subclasses of the `Gold` class representing various forms of treasure that can be collected.

Potion Decorator:

Handles the change of the players' attack and defence value within level. Implements the decorator pattern to dynamically modify player attributes without adding extra attributes in the player's base class or the need for explicit tracking of potions consumed in each level.

Main Logic:

The entry point and primary game loop, responsible for handling user input, updating the game state, and rendering the game board.

Direction Enum:

Defines possible movement directions for characters (e.g., North, South, East, West). Used to standardize movement commands and simplify direction handling within the game logic.

Design

The center of my design is using Object-Oriented Programming (OOP) fundamental concepts Inheritance and Polymorphism to achieve high cohesion and low coupling. I created two three-level inheritance hierarchies: one for characters and one for items.

Three-level inheritance hierarchy for characters:

In my three-level inheritance hierarchy for characters, the `Character` class acts as an abstract base class for both `Player` and `Enemy`. It defines common attributes and methods, including `attack`, `defend`, `move`, and `consume`. To enhance the flexibility of the `attack` and `defend` methods, the `Character` class incorporates `accuracy` and `agility` variables. These attributes enable subtle adjustments to the success rates of attacks and defenses, providing a more flexible approach to handling these actions. Both `Player` and `Enemy` extend `Character`, adding their own specific attributes and methods to cater to their unique roles. The `Player` class adds a `consume` method for handling potions and/or treasures, while the `Enemy` class sticks to the inherited methods. Concrete classes, such as `Vampire`, build directly on their parent class (`Player` in the case of `Vampire`) by customizing methods to incorporate their unique abilities. For instance, the `Vampire` class overrides the `attack` method to implement its unique healing ability.

This design promotes high cohesion, as each class is focused on a specific role and functionality, ensuring that its purpose and behavior are clearly defined. Furthermore, this

design maintains low coupling by ensuring that derived and concrete classes interact only with their parent class and not with other derived or concrete classes. Each class depends solely on its parent class, which simplifies interactions and reduces dependencies across the system. Moreover, this approach not only keeps everything well-organized but also makes it easy to add new types of players or enemies. New characters can be introduced simply by adding concrete subclass and adjusting methods as needed, making the system flexible and easy to grow.

Three-level inheritance hierarchy for items:

In my three-level inheritance hierarchy for items, the base class `Item` defines a pure virtual function called `consume`, ensuring that each specific item must provide its own implementation of how it is consumed. To enhance the flexibility and generality of the `consume` function, a `potionMult` variable is introduced. This variable allows subtle adjustments in the potions' effects, adding complexity and flexibility to how they are consumed. The `Potion` class includes a `getEffect` method to describe what each potion does, as different potions have different effects. Each concrete potion class implements the `consume` function based on its unique effect. In contrast, the `Gold` class implements `consume` by simply adding its value to the consumer's total gold. This design achieves high cohesion by ensuring that each class has a single, well-defined responsibility. The `Potion` class is responsible for potion-specific behaviors, while the `Gold` class handles gold-related logic. This focus makes each class easier to understand and maintain. At the same time, low coupling is maintained because each class only interacts through its parent class and doesn't rely on the details of other item types. New types of items, such as additional potions or treasures, can be added easily by adding concrete class and implementing the `consume` method as needed. This modularity ensures that changes in one class have minimal impact on others, making the system flexible and scalable.

Game:

The game board is represented by `activeFloor`, which tracks various elements in the game using a `vector<string>`. The active player is managed using a smart pointer to a `Player` object. Enemies are stored in `enemyList` (a `vector<Enemy>`), gold items are kept in `goldList` (a `vector<Gold>`), and potions are organized in `potionList` (a `vector<Potion>`). The stair is represented as a simple `pair`, indicating its position on the game board. The primary methods of the `Game` class include initializing the floor, generating the player, and placing stairs, potions, gold, and enemies. It also handles player movements, attacks, and item consumption. When initializing the floor, it's crucial to determine which chamber each tile belongs to. My approach involves manually creating a copy of the default floor text file and replacing the dots with numbers corresponding to chambers, from 1 to 5 (with the capability to accommodate up to 9). I then read this

modified text file and store it to `chamberFloor`. The chamber tiles are organized into `chamberList`, which is a `vector<vector<pair>>`.

As I read the copied floor text file, each tile numbered `i` (where `i` ranges from 1 to 9) is added to `chamberList[i-1]`. Consequently, the size of `chamberList` matches the number of chambers, and the size of `chamberList[i]` corresponds to the number of tiles in chamber `i`.

With this setup, the floor and chamber configurations can be easily updated by modifying the text file, without requiring a recompilation of the code. Furthermore, this approach makes it easy to ensure that “every chamber is equally likely” and that there is “equal probability for any floor tile within a chamber” when generating characters, stair and items. To achieve this, I first select a chamber by randomly generating a number from 0 to the total number of chambers minus one. Next, I pick a random tile within the chosen chamber by generating a number from 0 to the number of tiles in that chamber minus one. If the selected tile is suitable for placing a character, item or stair (with `chamberFloor` helping to quickly identify which chamber a tile belongs to), I place it there. If the tile isn’t suitable, I just pick again until I find a good spot. When enemies, potions, or gold are added or removed, I update their respective lists—`enemyList` for enemies, `potionList` for potions, and `goldList` for gold items. Additionally, I also update `activeFloor` to reflect these changes on the game board. This setup makes it easy to print the game board and check the eight adjacent tiles for any given position.

In the game, after the player’s turn, it’s necessary to notify all enemies to take their actions. While the Observer pattern is typically a good choice for this type of notification, it doesn’t fit well with our requirement for enemies to move in a specific line-by-line order. To ensure that enemies act in the correct sequence, I sort `enemyList` according to the required order before allowing each enemy to respond individually.

Design Patterns

1. *Decorator Pattern*: To handle temporary potion effects, the decorator pattern was employed. This pattern modifies the `getAtk` and `getDef` methods of the `Player` class by wrapping them with decorators that adjust the player’s attributes based on consumed potions. This eliminates the need for explicitly tracking which potions have been consumed on each level.

2. *Polymorphism*: Polymorphism is utilized extensively throughout the design to manage different types of characters and items. This approach allows for a uniform interface while enabling specific implementations for different subclasses.

3. *Static Vector for Merchants*: A static vector is used to keep track of all `Merchant` instances in the game. This design choice ensures that merchants are managed consistently and can be accessed globally within the game logic.

Changes from Original Design

Overall, I'm very satisfied with my original design. It made the implementation quite smooth, with only a few minor tweaks needed, as listed below:

1. *Enhanced Control Methods*: Additional control methods were integrated into the `Game` class to improve the game's functionality and control over various game elements.
2. *Refined Implementations*: Some overridden methods in concrete classes were removed and generalized in the base classes. This refactoring led to cleaner and more maintainable code by centralizing common functionality in the base classes.
3. *Smart Pointers*: The use of smart pointers was introduced to manage memory more effectively. This change, not included in the original UML design, simplifies memory management and mitigates risks of memory leaks and dangling pointers.

Specific Implementations

1. *Attack and Defend Methods*: Both `attack` and `defend` methods are implemented in the `Character` class, with specific behaviors defined in the subclasses (`Player` and `Enemy`).
2. *Consume and Consumed Methods*: Methods for handling consumption of items are implemented in both the `Player` and `Item` classes. This includes handling the effects of consumed potions and the management of item interactions.
3. *Merchant Management*: A static vector manages all `Merchant` instances, ensuring centralized control and tracking throughout the game's lifecycle.

By implementing these design elements and changes, the project achieves a more maintainable, scalable, and functional game architecture, improving overall code quality and game playability.

Resilience to Change

As discussed in Design part, the design of my three-level inheritance hierarchy for characters and items fosters high cohesion by ensuring each class has a specific role and functionality, making its purpose and behavior clear. This design also maintains low coupling by having derived and concrete classes interact solely with their parent class, without dependencies on other derived or concrete classes. This approach makes it straightforward to introduce new types of players, enemies, potions, and gold. Adding new characters or items involves simply creating a new concrete subclass and adjusting methods as needed, ensuring the system remains flexible and easy to extend.

Additionally, complementary methods like `attack` and `defend` or `consume` and `consumed` help to confine special interactions or effects to individual classes. For example, the `Player`'s `consume` method calls the `consumed` method of the `Item` class,

which manages all item effects within the item class itself. This design allows for adding or removing items without affecting the `Player` class. Similarly, special interactions, such as an orc dealing extra damage to a goblin, are handled by overriding the orc's `attack` method to check if the target is a goblin and apply additional damage if true. If the orc or goblin is removed, the condition simply isn't met, and no changes are required in other classes. This design approach also ensures minimal recompilation when modifications are made. My design also supports changes to the floor configuration by simply modifying the default floor text file. If the floor remains the same size and accommodates up to 9 chambers, the implementation will continue to work correctly without recompilation. If the floor size changes, updating two constants is required, which would need recompilation.

Answers to Questions

2.1 Player Character

How could you design your system so that each race could be easily generated?

Additionally, how difficult does such a solution make adding additional races?

To design a system where each race can be easily generated and new races can be added with minimal difficulty, I used inheritance, encapsulation, polymorphism and abstract base classes. This way common functionalities need to be implemented only once in the parent class and can be inherited by multiple subclasses. Subclasses can add their own specific behaviors or modify existing behaviors inherited from the parent class. This allows for easy extension of functionality without modifying the existing code in the parent class. In my design, the parent class `Player` provide common functionality of all races. Race specific behavior is contained in the subclass of specific race. As a result, to add a new race, I can simply create a new subclass of `Player` and put the race-specific abilities/behavior in the new class as needed. This design minimizes code duplication, maintains flexibility, and reduces the impact on existing code when adding or removing races.

2.2 Enemies

How does your system handle generating different enemies? Is it different from how you generate the player character? Why or why not?

My system handles generating different enemies in a manner very similar to generating different player races, leveraging inheritance, encapsulation, polymorphism, and abstract base classes. Both `Player` and `Enemy` classes inherit from a common abstract base class called `Character`, which defines shared attributes and behaviors. Specific enemy types and player races inherit from their respective `Player` and `Enemy` classes, ensuring unique attributes while sharing common attributes and behaviors. Polymorphism enables the game to manage different characters through generic `Character` pointers. Each concrete class constructor only requires parameters unique to its instantiation, simplifying creation. This approach minimizes code duplication and reduces compilation overhead by centralizing common attributes and behaviors in the `Character` base class, ensuring that changes only need to be updated once. The consistent approach to both player characters and enemies enhances maintainability, scalability, and efficiency.

How could you implement the various abilities for the enemy characters? Do you use the same techniques as for the player character races? Explain.

The implementation of abilities for enemy characters follows the same techniques as for player character races, leveraging inheritance, polymorphism, and encapsulation. Each ability is associated with a basic method in the `Character` class, such as `attack`, which is a common action for all characters. These methods have virtual implementations in the `Character` class, allowing characters without special abilities to use the default implementation. For characters with special abilities, such as the `Vampire` with regeneration after an attack, the specific class overrides the relevant method to include the unique behavior. This design keeps special abilities contained within the specific concrete player race or enemy type subclass, ensuring that added features do not affect the base class or other derived classes. It also simplifies the process of adding new type for both enemies and players, as you only need to create a new subclass that inherits from either `Player` or `Enemy` and override the necessary methods to implement the special abilities in the new subclass. This approach ensures maintainability, scalability, and the encapsulation of abilities.

2.3.1 Potions

What design pattern could you use to model the effects of temporary potions (Wound/Boost Atk/Def) so that you do not need to explicitly track which potions the player character has consumed on any particular floor?

To model the effects of temporary potions, I am using the decorator pattern, which modifies the `getAtk` and `getDef` methods of the `Player` class. The `Player` class holds a pointer to an abstract base class `Consumed`, which stores the base attack and defense

values. The concrete component, class `NoPotion`, simply returns these base values. The concrete decorators, class `AtkPot` and `DefPot`, wrap around the `NoPotion` component and modify the `getAtk` and `getDef` functions accordingly. This design allows the `atk` and `def` values to be dynamically altered by applying the decorators, which represent the consumed potions. Because the decorators handle the modifications to these values, there is no need to explicitly track which potions the player character has consumed on any floor. Each level reset involves clearing or reassigning the decorators, automatically reverting the player's stats to their base values without maintaining separate fields for each potion effect. This approach ensures a clean, modular way to manage temporary boosts and wounds, simplifying the code and reducing potential errors.

2.3.2 Treasure

How could you generate items so that the generation of Treasure and Potions reuses as much code as possible? That is, how would you structure your system so that the generation of a potion and then generation of treasure does not duplicate code?

To generate items such as Treasure and Potions while reusing as much code as possible, I similarly, structured my system using inheritance, encapsulation, polymorphism, and abstract base classes. Both `Gold` and `Potion` classes derive from a shared abstract base class named `Item`, which defines the common aspects of generating both `Gold` and `Potions`. This `Item` class includes the methods for determining the chamber and location for item generation, applying uniformly to all items. By centralizing these common functionalities within the `Item` base class, I eliminate code duplication and ensure that the generation process for both Treasure and Potions leverages the same underlying logic. This approach simplifies maintenance and allows for easy extension if new item types are introduced in the future.

Extra Credit Features

The extra credit feature I implemented was complete memory management using smart pointers. Although this feature was not part of my original UML design, I decided to incorporate it during the implementation phase after learning about smart pointers. The main advantage of using smart pointers was the ease of memory management. With smart pointers, I didn't have to manually manage memory allocation and deallocation, which eliminated the need to explicitly call `delete` and reduced the risk of memory leaks. This feature significantly simplified the memory management process and made the code more robust and maintainable. Contrary to my initial expectations, using smart pointers did not make the implementation more challenging. In fact, it streamlined the development process by automating memory management tasks, allowing me to focus more on the core functionality of the project.

Final Questions

What lessons I learned about writing large programs

From my experience with writing this large program by myself, I've learned that careful planning is crucial for a successful implementation. Investing time upfront to develop a thorough and detailed plan provides a clear roadmap, helping to avoid issues during the implementation phase and minimizing costly revisions. Although it was hard to resist the urge to jump straight into coding, I'm glad I took the time to consider all aspects thoroughly. It truly paid off, resulting in a smoother and more efficient implementation.

I also learned that applying fundamental Object-Oriented Programming (OOP) concepts, such as inheritance and polymorphism, can greatly simplify the design and implementation process. This approach leads to more successful and manageable outcomes. By applying these principles, I was able to develop a project that I am truly proud of.

What I would have done differently

Overall, I had a great experience and am really pleased with what I achieved in such a short time. The project progressed smoothly, and it was satisfying to see everything come together so well. I'm excited about what I learned and looking forward to using these insights in future projects.

If I had the chance to start over, I would apply the Model-View-Controller (MVC) pattern. It would provide a clearer structure and enhance the organization of the project. Additionally, due to time constraints, I began compiling/testing only after the full implementation was complete, which made it somewhat overwhelming to tackle and resolve all the errors at once. In retrospect, adopting an incremental development approach would have been beneficial.